Software Architecture and Design Specification

Project: Smart Expense & Budget Management Application

Version: 1.0

Authors: Viren James Fernandes (PES2UG24CS590), YV Prateek (PES2UG24CS617), Vikranth Acharyya (PES2UG24CS587), Vidushi Singh (PES2UG24CS581)

Date: 14-09-2026

Status: Draft

Team No: 3

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Team 3	Initial SAD draft derived from approved SRS v1.0 for the Smart Expense & Budget Management Application

Approvals

Role	Name	Signature/Date
Course Coordinator	Prof. Pavithra M	Pending

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Smart Expense & Budget Management Application. It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS v1.0) into a concrete architecture, component design, sequence flows, API contracts, and security model that development and QA teams can implement and verify against.

1.2 Scope

This document covers the architecture and design of user-facing services for income and expense tracking, budget creation and monitoring, savings goal management, recurring transactions, financial summaries and reports, and budget-related notifications. Consistent with the SRS, it does not cover direct banking transactions, loan processing, investment management, or financial transactions with external banks.

1.3 Audience

Developers, QA engineers, system integrators, maintenance technicians, and assessment evaluators.

1.4 Definitions

SRS, UI, API, DB, OTP, CRUD, NFR, FR, RTM, ADR (Architecture Decision Record), JWT (JSON Web Token), TLS (Transport Layer Security), WCAG (Web Content Accessibility Guidelines).

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables for the Smart Expense & Budget Management Application, including a component (UML) diagram, UML sequence diagrams for key flows, API design, error handling and monitoring strategy, UX design notes, and traceability back to the SRS requirements.

2.2 Related Documents

Software Requirements Specification (SRS v1.0) for the Smart Expense & Budget Management Application; Software Test Plan (STP); Requirements Traceability Matrix (RTM, SRS Section 8).

3. Architecture

3.1 Goals & Constraints

Goals: secure handling of users' personal financial data; reliable and consistent transaction and budget calculations; 99.9% monthly availability (SEA-NF-002); responsive performance for common operations (SEA-NF-001); accessibility conforming to WCAG 2.1 AA (SEA-NF-004).

Constraints: the system shall not directly perform banking transactions or access users' bank accounts unless an authorized banking API is explicitly integrated; requires network connectivity for cloud/server-based features; must maintain data consistency when transactions, budgets, or goals are added, modified, or deleted; must comply with applicable data privacy and security requirements.

3.2 Stakeholders & Concerns

- User: security of personal financial data, ease of recording transactions and budgets, timely and accurate alerts.
- Administrator: manageability of user accounts and transaction categories, visibility into system activity.
- Regulators/Privacy: data privacy and protection of financial information.
- Developers/Maintenance: modularity, maintainability, and testability of the system.

3.3 Component (UML) Diagram

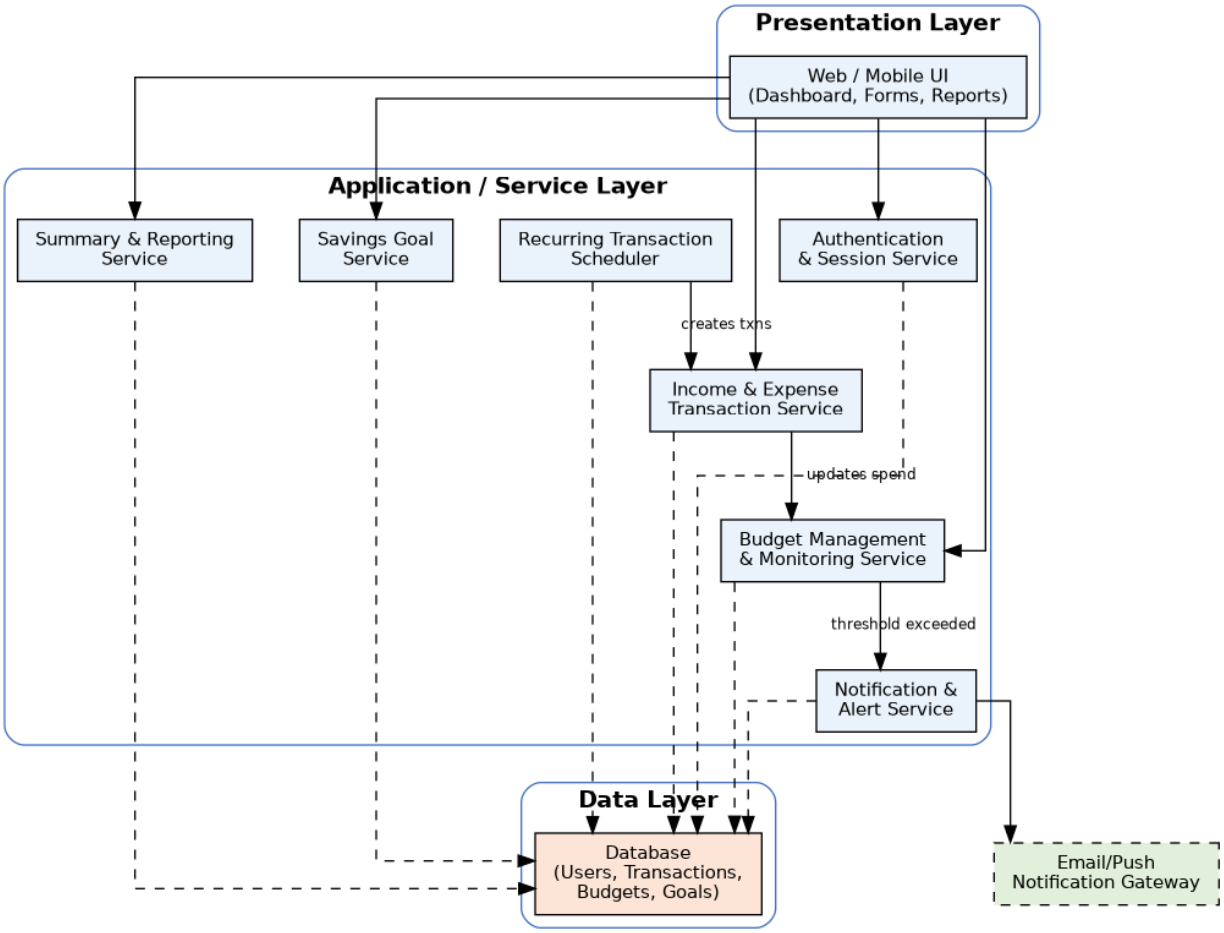


Figure 1: Component diagram showing the presentation, application/service, and data layers.

3.4 Component Descriptions

- Web/Mobile UI: Dashboard, transaction entry forms, budget and report views; supports readable text, clear labels, sufficient contrast, and keyboard navigation.
- Authentication & Session Service: user registration, login, password reset, profile updates, and session/token management (SEA-F-001..004, SEA-SR-005).
- Income & Expense Transaction Service: create, edit, and delete income/expense transactions; categorization; search and filter (SEA-F-005..007, SEA-F-013).
- Budget Management & Monitoring Service: budget creation, tracking spend against limits, and remaining-budget/available-balance calculation (SEA-F-008..010).
- Savings Goal Service: creation and management of savings goals and progress tracking (SEA-F-014).
- Recurring Transaction Scheduler: automatically generates recurring income and expense entries (SEA-F-016).
- Summary & Reporting Service: daily, weekly, and monthly summaries, graphical spending reports, and report export (SEA-F-011, SEA-F-012, SEA-F-017).
- Notification & Alert Service: generates and delivers alerts when spending approaches or exceeds budget limits (SEA-F-015).
- Database: stores users, transactions, budgets, categories, savings goals, and notifications.

3.5 Chosen Architecture Pattern and Rationale

A layered (n-tier) architecture is adopted, comprising presentation, application/service, and data layers. This provides a clear separation of concerns and maps naturally onto the SRS's modular requirement structure (Authentication Module, Income Module, Budget Module, Savings Goal Module, Recurring Transaction Module, Reporting Module, Notification Module per the RTM). Within the service layer, each capability is implemented as an independent service module communicating through well-defined internal APIs, giving a lightweight "modular monolith" that can be decomposed into microservices later if scale demands it. A full microservices architecture was rejected for the initial release as unnecessary operational complexity for a personal finance application of this scale.

3.6 Technology Stack & Data Stores

- Frontend: responsive Web/mobile client (e.g., React or equivalent) supporting keyboard navigation and screen readers per WCAG 2.1 AA.
- Backend: RESTful application server (e.g., Spring Boot or Node.js/Express) exposing JSON APIs over HTTPS.
- Database: relational DBMS (e.g., PostgreSQL/MySQL) for transactional integrity, with tables for Users, Transactions, Budgets, Categories, SavingsGoals, RecurringRules, and Notifications.
- Security: TLS 1.2+ in transit; passwords stored using a strong one-way hash (SEA-SR-002); JWT-based session tokens with inactivity-based expiry (SEA-SR-005).
- Notification: email/push notification gateway (optional third-party integration) for budget alerts.

3.7 Risks & Mitigations

- Risk: inconsistent budget totals under concurrent transaction updates → Mitigation: database transactions/row-level locking on budget updates.
- Risk: missed or duplicate recurring transactions → Mitigation: idempotent scheduler with an execution log.
- Risk: unauthorized cross-user data access → Mitigation: per-request authorization checks tied to the authenticated session (SEA-SR-003).
- Risk: notification delivery failure → Mitigation: retry queue with in-app alert fallback.

3.8 Traceability to Requirements

- SEA-F-001..004 → Authentication & Session Service
- SEA-F-005..007, SEA-F-013 → Income & Expense Transaction Service
- SEA-F-008..010 → Budget Management & Monitoring Service
- SEA-F-014 → Savings Goal Service
- SEA-F-016 → Recurring Transaction Scheduler
- SEA-F-011, SEA-F-012, SEA-F-017 → Summary & Reporting Service
- SEA-F-015 → Notification & Alert Service
- SEA-SR-001..005 → Security Architecture (cross-cutting)

3.9 Security Architecture

Threat Modeling (STRIDE):

- Spoofing → authentication via registered credentials and hashed passwords (SEA-F-002, SEA-SR-002).
- Tampering → server-side input validation on all API requests (SEA-SR-004).
- Repudiation → audit logging of authentication and account-management activity.
- Information Disclosure → HTTPS/TLS for all client-server traffic; passwords never stored in plaintext (SEA-SR-001, SEA-SR-002).
- Denial of Service → rate limiting on authentication endpoints.
- Elevation of Privilege → per-user authorization preventing cross-user access to transactions, budgets, and goals (SEA-SR-003); role separation between User and Administrator functions.

4. Design

4.1 Design Overview

The application is designed as layered modules matching the components in Section 3.4. Each module is independently testable and maintainable, and exposes REST APIs that are consumed by the presentation (UI) layer, in line with the maintainability goal in SEA-NF-005.

4.2 UML Sequence Diagrams

Two sequence diagrams are provided below, covering the core “add expense with budget alert” flow and the authentication flow.

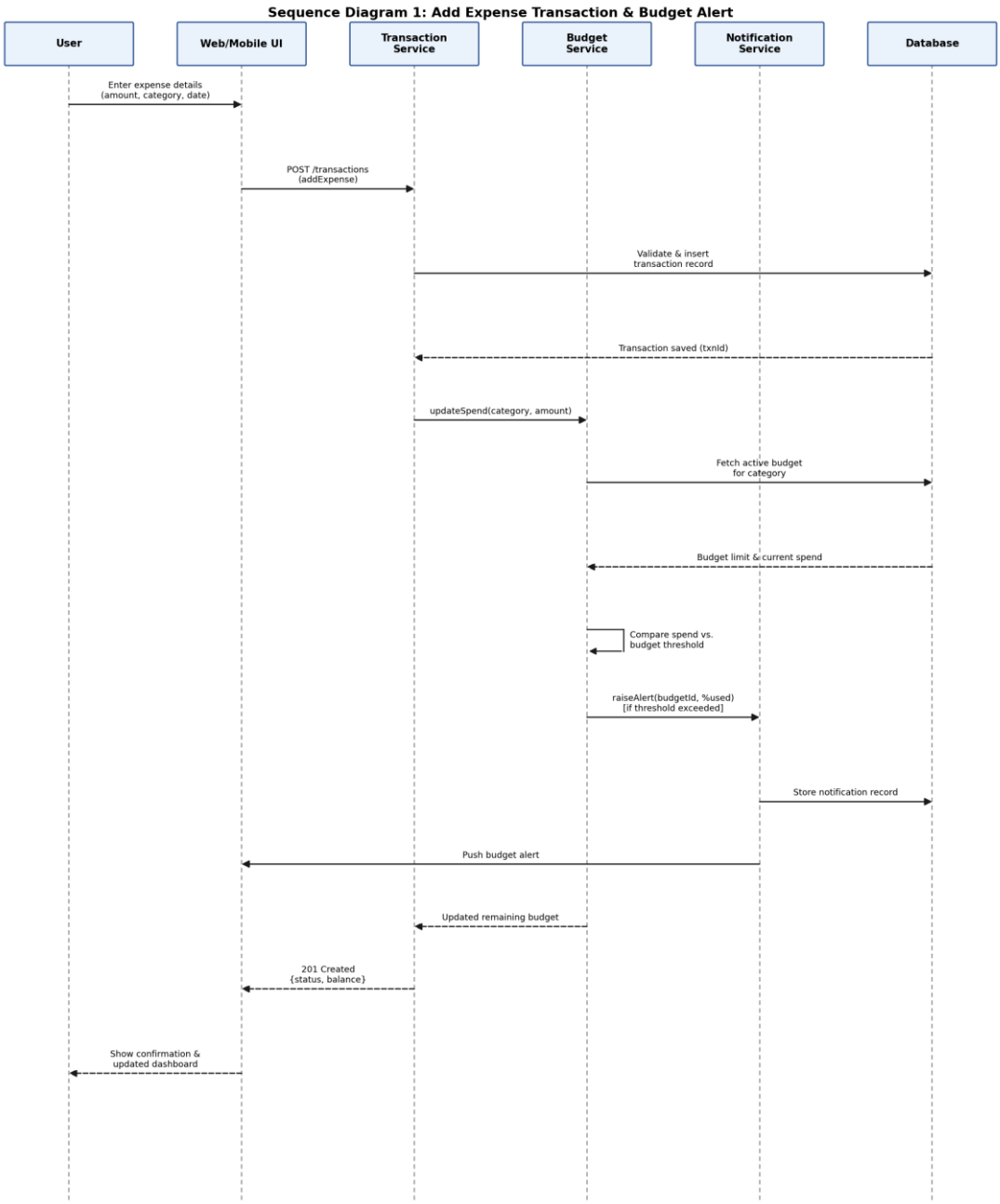


Figure 2: Sequence diagram – Add Expense Transaction & Budget Alert (SEA-F-005/006, SEA-F-009, SEA-F-015).

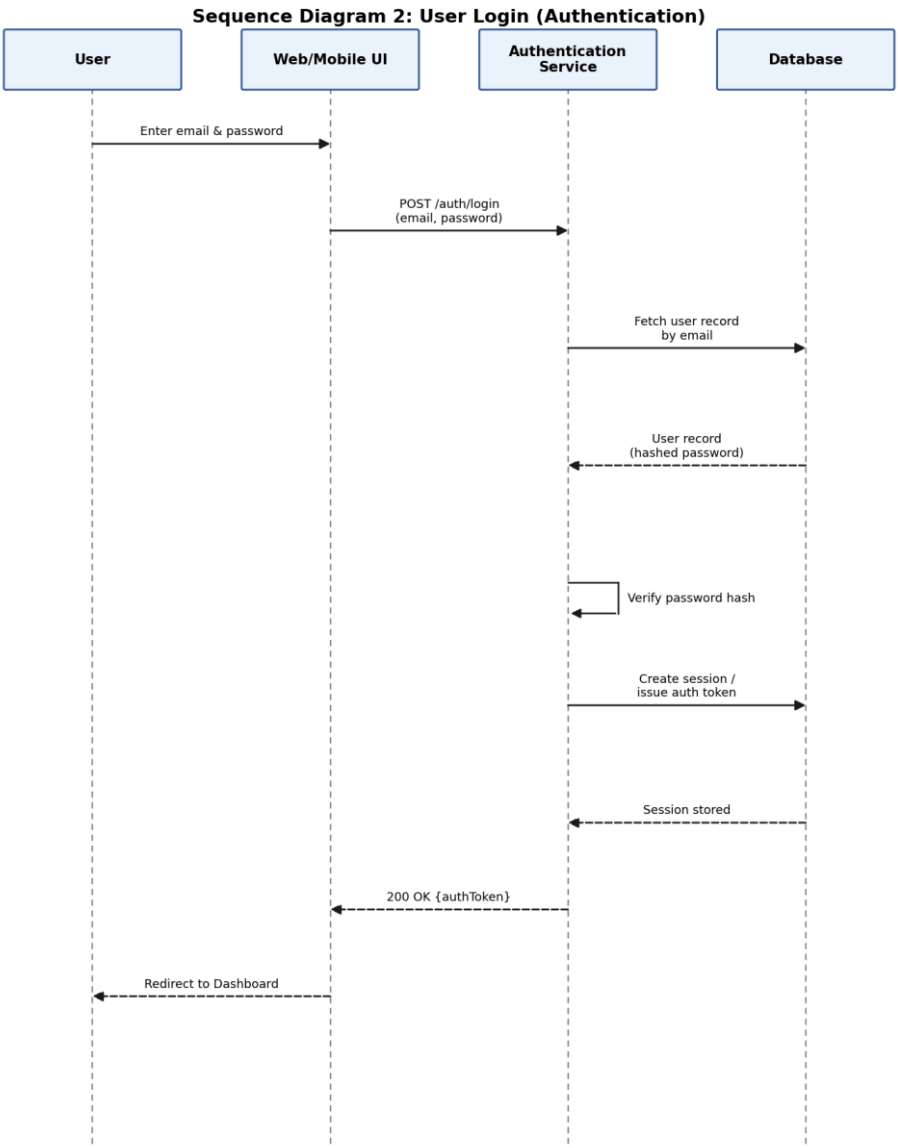


Figure 3: Sequence diagram – User Login / Authentication (SEA-F-002, SEA-SR-002).

4.3 API Design

Endpoint: POST /auth/login (Authentication & Session Service)

- Request: { email, password }
- Response: { authToken, expiresIn }
- Errors: 401 Invalid credentials, 429 Too many attempts

Endpoint: POST /transactions (Income & Expense Transaction Service)

- Request: { type: "expense" | "income", amount, categoryId, date, note }
- Response: { transactionId, status, remainingBudget }
- Errors: 400 Invalid input, 401 Unauthorized, 404 Category not found

4.4 Error Handling, Logging & Monitoring

All APIs return a standardized JSON error envelope of the form { code, message }. No sensitive information (passwords, auth tokens, full card/account numbers) is written to logs. Monitoring covers API response time against the 5-second, 90th-percentile target (SEA-NF-001), authentication failure rate, transaction failure rate, budget-alert delivery latency, and monthly uptime against the 99.9% target (SEA-NF-002).

4.5 UX Design

The dashboard presents total income, total expenses, and remaining budget at a glance. The UI uses accessible font sizes, high-contrast text, clear category labels and icons, keyboard navigation, and graphical (chart-based) views of spending patterns, consistent with SEA-NF-004.

4.6 Open Issues & Next Steps

- Bank-account synchronization via an authorized banking API (out of scope for the current release).
- AI-assisted category suggestions for new transactions.
- Multi-currency support.
- Shared/family budgets with multiple contributing users.

5. Appendices

5.1 Glossary

SRS, UI, API, DB, OTP, CRUD, NFR, FR, RTM, TLS, JWT, WCAG.

5.2 References

IEEE 42010 (Architecture Description), OWASP Application Security Verification Standard (ASVS), NIST SP 800-63B (Digital Identity Guidelines – password/authentication), WCAG 2.1.

5.3 Tools

Graphviz and Python (component and sequence diagrams), draw.io, Swagger/OpenAPI (API design), Postman (API testing).